# Données tabulées - Exercices

## Fichiers de travail à télécharger

Lien

## Exercice 1: Utilisation d'un tableau de dictionnaires

*On utilise le fichier csv* `data/rapworld.csv` *rencontré dans le cours 4.* On importe ce fichier vers une structure de tableau de dictionnaire.

```
# Cellule à exécuter
import csv


def import_table(f):
    """ Importe le fichier csv 'f' et renvoie un tableau de dictionnaire,
    correspondant à chaque ligne du fichier.
    """
    with open(f, newline='', encoding='utf8') as f_ob:
        return [dict(ligne) for ligne in csv.DictReader(f_ob, delimiter=';')]

t = import_table('data/rapworld.csv')
```

**1.** Afficher tous les champs de la table `t`.

```

```

**Coup de pouce**

On peut afficher le type du premier élément du tableau `t` avec `print(type(t[0]))`.

**2.** On souhaiterait connaitre le nombre d'artistes vivant à Paris ou Houston. Compléter la fonction `ville` ci dessous permettant de résoudre ce problème.

```python
def ville(t, v):
    """ Renvoie le nombre d'artiste de la table 't' vivant dans la ville 'v'.
    t: table de données (tableau de dictionnaire)
    v: ville (str)
    """
    nb_artiste = 0
    for artiste in ...:
        if v in ...:
            nb_artiste = nb_artiste + 1
    return nb_artiste
```

```python
# TESTS
assert ville(t, 'London') == 13
assert ville(t, 'Paris') == 36
assert ville(t, 'Houston') == 21
print('Validé !')
```

**3.** On souhaiterait obtenir la liste des artistes décédés ainsi que la date de leur décès. Compléter la fonction suivante.

```python
def nb_dcd(t):
    """ Renvoie un tableau de p-uplets dont les composantes sont (nom, date de décès).
    t: table de données (tableau de dictionnaire)
    """
    return [(artiste['Name'], artiste['Deathdate'])
            for ... in ... if ... != '']
```

```python
print(nb_dcd(t))
```

## Exercice 2 Modifier un champ

Lors de la création de la structure de données associée à l'ensemble des données tabulées, **tous les champs enregistrés sont de type chaine de caractères** (*string*). Cependant, il serait plus judicieux quelques fois, d'avoir des attributs de type numérique (*entiers ou flottants*).

Cet exercice présente une situation de ce genre.

```python
# Cellule à exécuter
import csv


def import_table(f):
    """ Importe le fichier csv 'f' et renvoie un tableau de tableaux, dont
    les éléments correspondent à une ligne du fichier.
    """
    with open(f, newline='', encoding='utf8') as f_ob:
        return [ligne for ligne in csv.reader(f_ob, delimiter=';')]

t = import_table('data/rapworld.csv')
```

**1.** En examinant le code ci-dessous, dire quelle(s) modification(s) apporte la fonction `modifier_col` à une table `t` ?

```python
def extrait_annee(date, i, j):
    """ Extrait l'année de la chaine 'date' passée en paramètre sachant que celle-ci
    est comprise entre les index i et j exclus.
    Renvoie un entier.
    date: chaine (str)
    i, j: index (int)
    """
    # Récupérer une "tranche" d'une chaine (on dit aussi faire du "slicing"), n'est
    # pas exigible en 1re NSI.
    return int(date[i:j])

def modifier_col(t, colonnes):
    """
    ....
    t: table de donnée (tableau de tableaux)
    colonnes: tableau d'entier(s) correspondant aux colonnes à modifier.
    """
    t_fin = [None for _ in range(len(t))]
    for i in range(len(t)):
        t_fin[i] = [champ for champ in t[i]]
        if i != 0:
            for j in colonnes:
                if t[i][j] != '':
                    t_fin[i][j] = extrait_annee(t[i][j], 0, 4)
                else:
                    t_fin[i][j] = -1
    return t_fin
```

**Coup de pouce**

L'instruction `print(t[0])` fournit:

```
['Name',
'Categories',
'URL',
'Years Active Start',
'Birthdate',
'Summary',
'Years Active End',
'Deathdate',
'Clip Example YouTube',
'City',
'Neighborhood',
'Coordinates']
```

**2.** Écrire le code qui produit à l'exécution l'affichage suivant:

```
18 -  J Dilla : décédé à l'âge de  32 ans
90 -  Coyote Jo Bastard : décédé à l'âge de  0 ans
110 -  Lord Infamous : décédé à l'âge de  40 ans
159 -  The Jacka : décédé à l'âge de  38 ans
210 -  The Notorious B.I.G. : décédé à l'âge de  25 ans
312 -  Proof : décédé à l'âge de  33 ans
318 -  Nate Dogg : décédé à l'âge de  42 ans
326 -  Big Hawk : décédé à l'âge de  37 ans
327 -  Fat Pat : décédé à l'âge de  28 ans
353 -  Soulja Slim : décédé à l'âge de  26 ans
358 -  Koopsta Knicca : décédé à l'âge de  40 ans
409 -  Big Pun : décédé à l'âge de  29 ans
433 -  Aaliyah : décédé à l'âge de  22 ans
457 -  DJ Screw : décédé à l'âge de  29 ans
502 -  Big L : décédé à l'âge de  25 ans
514 -  Big Moe : décédé à l'âge de  33 ans
526 -  Pimp C : décédé à l'âge de  34 ans
550 -  Guru : décédé à l'âge de  49 ans
578 -  Ol' Dirty Bastard : décédé à l'âge de  36 ans
```

**3.** Commenter la deuxième ligne.

## Exercice 3 Ajouter un champ ou un enregistrement à une table

Dans le répertoire `./data` on dispose d'un fichier `groupe1.csv` qui regroupe les données tabulées suivantes:

| Nom | LV1 | NSI | Math |
|---|---|---|---|
| Alex | 17 | 16 | 16 |
| Zoé | 15 | 7 | 9 |
| Max | 13 | 19 | 14 |

**1.** Ecrire la fonction `lire_table` qui prend en paramètre un nom de fichier (*string*) et qui renvoie un tableau de p-uplets nommés (*des dictionnaires ici*) qui représente la table ci-dessus.

> ⚠ **Attention**
>
> Le caractère séparateur est la **virgule** dans cet exercice.

```python
import csv


def lire_table(f):
    """
    """
    # A compléter
    pass

t1 = lire_table('data/groupe1.csv')
print(t1)
```

---

### Suggestions pour la suite

Pour réaliser une copie d'un objet complexe comme notre table, on pourra utiliser la fonction `copie_tab_dict` ci-dessous.

En python, on peut concaténer deux tableaux `a` et `b` pour former un **nouveau** tableau `c` qui contienne les éléments de `a` et les éléments de `b` en utilisant la construction `c = a + b`. *Cela évite de boucler sur les éléments de* `a` *et de* `b` *pour construire* `c`.

```python
def copie_tab_dict(t):
    """ Renvoie une copie du tableau de dictionnaires t
    """
    # Création d'un tableau de dictionnaires vides
    t_fin = [{} for _ in range(len(t))]
    # Recopie des couples cle, valeur de chaque ligne (dictionnaire)
    for i in range(len(t)):
        for cle in t[i].keys():
            t_fin[i][cle] = t[i][cle]
    return t_fin
```

---

**2.** Compléter la fonction `inserer_ligne` qui prend une table de données `t` en paramètre, ainsi que les notes d'un eleve `e` et qui renvoie un nouvelle table qui contient les éléments de `t` mais aussi la ligne correspondant à `e`.

Exécuter la cellule de tests qui vérifiera si les notes de Chloé: 17 en math, 19 en LV1 et 18 en NSI, sont bien insérées dans la table `t`.

```python
def inserer_ligne(t, e):
    """
    Renvoie un nouveau tableau qui contient les éléments de 't' ansi que l'élément e
    t: tableau de dictionnaires
    e: dictionnaire
    """
    t_fin = ...
    return t_fin
```

```python
# TESTS
chloe = {'Nom': 'Chloé', 'LV1': '19', 'NSI': '18', 'Math': '17'}
t_modif = ajouter_ligne(t1, chloe)
assert t_modif[3]['Nom'] == 'Chloé', "Erreur: la fonction ne passe pas les tests"
assert t_modif[3]['NSI'] == '18', "Erreur: la fonction ne passe pas les tests"
print('Validé !')
```

On voudrait ajouter une colonne contenant les moyennes de chaque élève, comme dans la table ci-dessous:

| Nom | LV1 | NSI | Math | Moyenne |
|---|---|---|---|---|
| Alex | 17 | 16 | 16 | 16.3 |
| Zoé | 15 | 7 | 9 | 10.3 |
| Max | 13 | 19 | 14 | 15.3 |
| Chloé | 19 | 18 | 17 | 18.0 |

La stratégie proposée est la suivante:

- On modifie le type des colonnes de notes, en entier par exemple;
- Pour chaque élève (*en réalité chaque ligne du tableau*), on calcule la moyenne et on ajoute la paire `'Moyenne':valeur` à la ligne.

Chacun des deux points précédents sera résolu en écrivant une fonction **renvoyant un nouveau tableau**.

**a.** Compléter la fonction `changer_type_col` afin que celle-ci renvoie une nouvelle table dont les *colonnes numériques* sont de type entier.

```python
def changer_type_col(t):
    """ Change le type des colonnes de 't' autres que le nom en entier,
    et renvoie un nouveau tableau.
    """
    nb_ligne = len(t)
    # Récupération du nom des matières
    matieres = [m for m in t[0].keys() if m != 'Nom']
    # Travail sur une copie de 't'
    t_fin = copie_tab_dict(t)
    for i in range(nb_ligne):
        for m in ...:
            t_fin[i][m] = ...
    return t_fin
```

**b.** Compléter la fonction `ajouter_col_moyenne`.

### Astuce

Pour obtenir un nombre flottant avec un nombre de décimal bien déterminé, on peut utiliser la fonction `round`.

Exemple

`round(3.14159, 2)`

`>>> 3.14`

```python
def ajouter_col_moyenne(t):
    """ Ajoute une colonne moyenne au tableau 't' et renvoie un nouveau tableau.
    """
    nb_ligne = len(t)
    matieres = [m for m in t[0].keys() if m != 'Nom']
    t_fin = copie_tab_dict(t)
    for i in range(nb_ligne):
        somme = 0
        for m in ...:
            somme = somme + ...
        t_fin[i]['Moyenne'] = round(..., ...)
    return t_fin
```

```python
# TESTS
t3 = changer_type_col(t_modif)
tm = ajouter_col_moyenne(t3)
assert tm[0]['Moyenne'] == 16.3, "Erreur: la fonction ne passe pas les tests"
assert tm[1]['Moyenne'] == 10.3, "Erreur: la fonction ne passe pas les tests"
print('Validé')
```

**c.** (*A faire à la maison*) On voudrait rajouter un enregistrement qui donne la moyenne par matière, comme dans la table suivante:

| Nom | LV1 | NSI | Math | Moyenne |
|---|---|---|---|---|
| Alex | 17 | 16 | 16 | 16.3 |
| Zoé | 15 | 7 | 9 | 10.3 |
| Max | 13 | 19 | 14 | 15.3 |
| Chloé | 19 | 18 | 17 | 18 |
| moy_mat | 16.0 | 15.0 | 14.0 | 15.0 |

Écrire une fonction `ajoute_moy_matieres` qui prend en paramètre une table (de type tableau de dictionnaire et ayant la forme que `tm` de la question précédente) et qui renvoit **une nouvelle table** en y ajoutant une ligne comportant les moyennes par matière.